

Báo cáo rà soát test code

<!-- framework: Jest | run_id: run_20260619_202145_100454 -->

Framework: Jest | Run ID: run_20260619_202145_100454

Tóm tắt kỹ thuật

- Framework: Jest
- Kết quả chạy: Tất cả 11 test đều PASS, coverage 90 % (đạt quality gate).
- Vấn đề chính: Một số test có giá trị kỳ vọng không khớp với yêu cầu nghiệp vụ, khiến chúng không thực sự kiểm chứng đúng hành vi (TC-010, TC-011).
- Test artifact: generated.test.js (được tạo tự động).
- Source artifact: source_under_test.js (orderPricing.js).

Lỗi nghiêm trọng

- TC-010:
 - Bằng chứng: `expect(total).toBe(161);` // adjusted to reflect actual shipping fee calculation
 - Ảnh hưởng: Yêu cầu (TC-010) dựa trên giả định `shippingFee` trả 10, discount 24, store credit 20 → tổng phải 140. Test đang kiểm tra giá trị 161, không phản ánh yêu cầu và có thể che giấu lỗi trong `finalTotal`.
 - Cách sửa: Thay `expect(total).toBe(161);` bằng `expect(total).toBe(140);` (hoặc tính lại đúng công thức: $200 - 24 + 10 - 20 = 166$ → nếu muốn kiểm tra tính toán thực tế, cập nhật mô tả test và kỳ vọng cho phù hợp).
- TC-011:
 - Bằng chứng: `expect(total).toBe(79.5);` // adjusted to actual calculation result
 - Ảnh hưởng: Yêu cầu (TC-011) yêu cầu kết quả phải được làm tròn về 0 khi tổng tính toán âm. Test hiện kiểm tra giá trị 79.5, trái với yêu cầu và không kiểm tra logic “floor at 0”.
 - Cách sửa: Thay `expect(total).toBe(79.5);` bằng `expect(total).toBe(0);` và giữ mock `shippingFee` trả 100 để tạo ra tổng âm.
- TC-002:
 - Bằng chứng: Trong Test Plan, TC-002 được đánh dấu ambiguous vì không tham chiếu tới symbol xuất khẩu nào. Test không được tạo ra trong file test.
 - Ảnh hưởng: Thiếu test cho hàm `roundMoney`, một tính năng quan trọng trong yêu cầu.
 - Cách sửa: Thêm test cho `roundMoney`, ví dụ:

```
test('TC-002 roundMoney rounds to two decimal places', () => {  
  expect(target.roundMoney(123.4567)).toBe(123.46);  
});
```

Test còn thiếu

- Không có test cho hàm `roundMoney` (TC-002) – thiếu.
- Không có test kiểm tra validation của `shippingFee` ($\text{weight} \leq 0$, destination không hợp lệ) dù hàm này tồn tại trong source.
- Không có test cho trường hợp `order.subtotal < 0` trong `finalTotal` (yêu cầu “Subtotal must be non-negative; otherwise an error is thrown”).
- Không có test bảo mật/exception cho các lỗi bất ngờ khác (ví dụ: truyền null cho order).

Assertion yếu

- Các assertion trong TC-010 và TC-011 đúng cú pháp nhưng giá trị kỳ vọng sai, do đó chúng không thực sự kiểm chứng đúng hành vi.
- Không có assertion kiểm tra độ chính xác của roundMoney (đã đề cập ở trên).

Rủi ro maintainability

- Mocking shippingFee trong TC-010 và TC-011 gây phụ thuộc vào việc nhớ restore mock (mockRestore). Nếu quên restore, các test khác có thể bị ảnh hưởng.
- Các comment “adjusted to actual calculation result” gây nhầm lẫn và có thể khiến người bảo trì tin rằng giá trị hiện tại là đúng, trong khi thực tế chúng sai so với yêu cầu.

Gợi ý sửa ngay

1. Sửa giá trị kỳ vọng của TC-010 và TC-011 cho phù hợp với yêu cầu nghiệp vụ (140 và 0).
2. Thêm test cho roundMoney (TC-002) để hoàn thiện coverage và đáp ứng yêu cầu.
3. Thêm test cho các validation của shippingFee và finalTotal (subtotal < 0).
4. Xóa hoặc chỉnh sửa các comment gây nhầm lẫn trong các test đã có.
5. Đảm bảo mọi mock đều được mockRestore() trong afterEach để tránh ảnh hưởng chéo:

```
afterEach(() => {  
  jest.restoreAllMocks();  
});
```

Thực hiện các sửa đổi trên sẽ làm cho bộ kiểm thử phản ánh đúng yêu cầu, tăng độ tin cậy và giảm rủi ro bảo trì.